

A

Project Report

Submitted in partial fulfilment of the requirements for the completion of the Course

EDS Lab Codetantra

Practical No.1

(2304102L)

Submitted by,

Payal Ankush Ghule -202501090088

Under the Guidance of
Ms. Savita Mane(SM)



(An Autonomous Institute Affiliated to Savitribai Phule Pune University)

Department of Mechanical Engineering,

MIT Academy of Engineering,

Alandi road, Pune - 412 105,

Maharashtra (India)

1.2.2. Fibonacci Series

32:17

Write a Python program that uses recursion to print the first terms of the Fibonacci series.

Input Format:

A single integer representing the number of terms to generate.

Output Format:

A single line containing the first terms of the Fibonacci sequence, separated by spaces.

```
print(f"Grade: {grade}")
```

Average time
0.006 s
6.20 ms

Maximum time
0.017 s
17.00 ms

5 out of 5 shown test case(s) passed

5 out of 5 hidden test case(s) passed

Test case 1 17 ms Debug

Expected output

4

55 65 78 89

Aggregate Percentage: 71.75

Grade: First Division

Actual output

4

55 65 78 89

Aggregate Percentage: 71.75

Grade: First Division

Test case 2 7 ms Debug

Expected output

5

56 78 97 86 93

Aggregate Percentage: 82.00

Grade: Distinction

Actual output

5

56 78 97 86 93

Aggregate Percentage: 82.00

Grade: Distinction

```
# Input number of
courses n = int(input())

#Input marks
marks = list(map(int, input().split()))

#Check if any subject is failed
if any(mark < 40 for mark in marks):
    print("Fail")
else:
    # Calculate aggregate percentage
    aggregate = sum(marks) / n

    # Print aggregate percentage formatted to two decimal places
    print(f"Aggregate Percentage: {aggregate:.2f}")

# Determine
grade if aggregate
> 75:
    grade = "Distinction"
elif 60 <= aggregate <
75:
    grade = "First Division"
elif 50 <= aggregate < 60:
    grade = "Second Division"
elif 40 <= aggregate < 50:
    grade = "Third Division"
```

1.2.1. Pass or

Fail 01:11

Write a Python program that accepts the number of courses and the marks of a student in those courses.

The grade is determined based on the aggregate percentage:

If the aggregate percentage is greater than 75, the grade is Distinction.

If the aggregate percentage is greater than or equal to 60 but less than 75, the grade is First Division.

If the aggregate percentage is greater than or equal to 50 but less than 60, the grade is Second Division.

If the aggregate percentage is greater than or equal to 40 but less than 50, the grade is Third Division.

Input Format:

The first input will be an integer , the number of courses.

The second input will be integers representing the marks of the student in each of the courses, separated by a space.

Output Format:

If the student passes all courses:

Print the aggregate percentage (formatted to two decimal places).

Print the grade based on the aggregate percentage.

If the student fails any course (marks < 40 in any course), print:

"Fail".

Note:

Aggregate Percentage: Average performance across all courses, calculated by summing all marks and dividing by the number of courses.

for i in range(1,11):

print(f"{n} x {i} = {n * i}")

The screenshot shows a code editor with a file named 'multiplica...'. The code is as follows:

```
1 n=int(input())
2 for i in range(1,11):
3     print(f"{n} x {i} = {n * i}")
```

Below the code editor, the execution results are displayed. The average time is 0.004 s and the maximum time is 0.006 s. The status indicates that 2 out of 2 shown test case(s) passed and 2 out of 2 hidden test case(s) passed.

Test case 1

Expected output	Actual output
8	8
8 x 1 = 8	8 x 1 = 8
8 x 2 = 16	8 x 2 = 16
8 x 3 = 24	8 x 3 = 24
8 x 4 = 32	8 x 4 = 32
8 x 5 = 40	8 x 5 = 40
8 x 6 = 48	8 x 6 = 48
8 x 7 = 56	8 x 7 = 56
8 x 8 = 64	8 x 8 = 64
8 x 9 = 72	8 x 9 = 72
8 x 10 = 80	8 x 10 = 80

Test case 2

Expected output	Actual output
5	5
5 x 1 = 5	5 x 1 = 5
5 x 2 = 10	5 x 2 = 10
5 x 3 = 15	5 x 3 = 15
5 x 4 = 20	5 x 4 = 20
5 x 5 = 25	5 x 5 = 25
5 x 6 = 30	5 x 6 = 30
5 x 7 = 35	5 x 7 = 35
5 x 8 = 40	5 x 8 = 40
5 x 9 = 45	5 x 9 = 45
5 x 10 = 50	5 x 10 = 50

At the bottom, there are buttons for 'Terminal', 'Test cases', 'Prev', 'Next', 'Reset', 'Submit', and 'Exit'.

```

from datetime import date

date1_input = input().strip()
date2_input = input().strip()

y1, m1, d1 = map(int, date1_input.split('-'))
y2, m2, d2 = map(int, date2_input.split('-'))

date1 = date(y1, m1, d1)
date2 = date(y2, m2, d2)

difference = date2 - date1

print(difference.days)

```

noOfDay...

```

1 from datetime import date
2 date1_input = input().strip()
3 date2_input = input().strip()
4 y1, m1, d1 = map(int, date1_input.split('-'))
5 y2, m2, d2 = map(int, date2_input.split('-'))
6 date1 = date(y1, m1, d1)
7 date2 = date(y2, m2, d2)
8 difference = date2 - date1
9 print(difference.days)

```

Average time: 0.007 s (7.25 ms) | Maximum time: 0.010 s (10.00 ms)

2 out of 2 shown test case(s) passed
2 out of 2 hidden test case(s) passed

Test case 1 (10 ms) [Debug]

Expected output	Actual output
2014-07-02	2014-07-02
2014-11-02	2014-11-02
123	123

Test case 2 (6 ms) [Debug]

Expected output	Actual output
2014-07-02	2014-07-02
2014-07-11	2014-07-11
9	9

Terminal | Test cases

< Prev | Reset | Submit | Next >

1.1.3. Days between Two

Dates 05:19

Write a Python program to calculate number of days between two dates.

Input Format:

The first line contains the first date in the format .

The second line contains the second date in the format .

Output Format:

An integer representing the number of days between the given dates.

Note:

The first date should always be considered earlier than the second date.

1.2.3. Pattern - 1

00:50

Write a Python program to print a pattern of asterisks in the form of a right-angled triangle.

Input Format:

The input is an integer, representing the number of rows in the pattern.

Output Format

The output should display the pattern of asterisks (*), with each row containing an increasing number of asterisks.

Note: Refer to the displayed test cases for the sample pattern.

```

def fibonacci(n):
    if n==1:
        return
    elif n==2:
        return 1
    else:
        return fibonacci(n-1) + fibonacci(n- 2)

n = int(input())
for i in range(1, n + 1):
    print(fibonacci(i), end=" ")

```

Average time: 0.011 s
10.75 ms

Maximum time: 0.028 s
28.00 ms

2 out of 2 shown test case(s) passed
2 out of 2 hidden test case(s) passed

Test case 1 4 ms Debug

Expected output: 5
0 1 1 2 3

Actual output: 5
0 1 1 2 3

Test case 2 3 ms Debug

Expected output: 15
0 1 1 2 3 5 8 13 21 34 55 89 144 233 377

Actual output: 15
0 1 1 2 3 5 8 13 21 34 55 89 144 233 377

Terminal Test cases

< Prev Reset Submit Next >

```

n = int(input())
for i in range(1, n + 1):
    for j in range(1, i + 1):
        print(j, end=" ")
    print()

```

numberP...

```

1 n = int(input())
2 for i in range(1, n + 1):
3     for j in range(1, i + 1):
4         print(j, end=" ")
5     print()

```

Average time: 0.004 s (3.75 ms) | Maximum time: 0.005 s (5.00 ms)

2 out of 2 shown test case(s) passed
2 out of 2 hidden test case(s) passed

Test case 1 (4 ms) [Debug] [Test]

Expected output	Actual output
5	5
1	1
1 2	1 2
1 2 3	1 2 3
1 2 3 4	1 2 3 4
1 2 3 4 5	1 2 3 4 5

Test case 2 (3 ms) [Debug] [Test]

Expected output	Actual output
8	8
1	1
1 2	1 2
1 2 3	1 2 3
1 2 3 4	1 2 3 4
1 2 3 4 5	1 2 3 4 5
1 2 3 4 5 6	1 2 3 4 5 6
1 2 3 4 5 6 7	1 2 3 4 5 6 7
1 2 3 4 5 6 7 8	1 2 3 4 5 6 7 8

Terminal [Test cases]

< Prev Reset Submit Next >

1.2.4. Pattern - 2

01:10

Write a Python program to print a right-angled triangle pattern of numbers.

Input Format:

The input is an integer, representing the number of rows in the pattern.

Output Format:

The output should display the pattern of numbers separated by space, with each row containing increasing numbers starting from 1 up to the row number.

Note:

Refer to the displayed test cases for the sample pattern.

```

n = int(input())
for i in range(1, n + 1):
    for j in range(i):
        print("*", end=" ")
    print()

```

The screenshot shows a coding platform interface for a problem titled "rightangl...". The code in the editor is as follows:

```

1 n = int(input())
2 for i in range(1, n + 1):
3     for j in range(i):
4         print("*", end=" ")
5     print()

```

The execution results show that the program passed all test cases:

- Average time: 0.006 s (6.00 ms)
- Maximum time: 0.008 s (8.00 ms)
- 2 out of 2 shown test case(s) passed
- 4 out of 4 hidden test case(s) passed

Two test cases are detailed below:

Test Case	Expected Output	Actual Output
Test case 1 (7 ms)	<pre> 5 * * * * * * * * * * * * * * * </pre>	<pre> 5 * * * * * * * * * * * * * * * </pre>
Test case 2 (6 ms)	<pre> 4 * * * * * * * * * * </pre>	<pre> 4 * * * * * * * * * * </pre>

The interface also includes a "Terminal" tab and a "Test cases" tab at the bottom. Navigation buttons at the bottom right include "< Prev", "Reset", "Submit", and "Next >".